

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

倍增与 ST 表

Binary Lifting and Sparse Table

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia

2026 年 10 月 30 日

目录

1. 倍增思想
2. 快速幂
3. 线性跳表
4. 树上倍增
5. ST 表
6. 例题精讲

目录

⑦ 总结

目录

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

1. 倍增思想
2. 快速幂
3. 线性跳表
4. 树上倍增
5. ST 表
6. 例题精讲

目录

7 总结

什么是倍增

倍增法：一种以 2 的幂（1, 2, 4, 8, . . . ）为步长向前跳跃的算法思想。

两个关键前提

- 问题具有**可合并性**：两个小区间的答案能拼成大区间的答案
- 目标规模可以用二进制拆分：任意正整数 n 都能写成若干 2 的幂之和

复杂度收益：把逐位累加的线性过程，压缩成二进制位数的对数过程。

$$\text{倍增} = \text{二进制拆分} + \text{跳跃式合并}$$

二进制拆分

以 $n = 13$ 为例：$13 = 8 + 4 + 1 = 2^3 + 2^2 + 2^0$。

位序号	2^3	2^2	2^1	2^0
权重	8	4	2	1
二进制位	1	1	0	1
贡献	8	4	0	1
累计	8	12	12	13

观察：从高位到低位扫描，遇到 1 就把当前权重计入累计。

推广：任意 n 只需要 $\lfloor \log_2 n \rfloor + 1$ 位，跳跃步数从 n 降到 $\log_2 n$。

倍增的三种典型形态

同一思想，三种载体

- 数值上的倍增：快速幂，底数不断平方
- 数组上的倍增：线性跳表，位置不断按 2 的幂跳跃
- 区间上的倍增：ST 表，长度翻倍合并

统一套路

- 定义 $f[i][j]$：以 i 为起点、规模为 2^j 的状态
- 边界：$f[i][0]$ 直接由输入得到
- 转移：$f[i][j]$ 由 $f[\cdot][j-1]$ 两块拼成
- 查询：把目标规模二进制拆分，逐位跳

先建表，再按二进制位跳

目录

1 倍增思想

2 快速幂

3 线性跳表

4 树上倍增

5 ST 表

6 例题精讲

目录

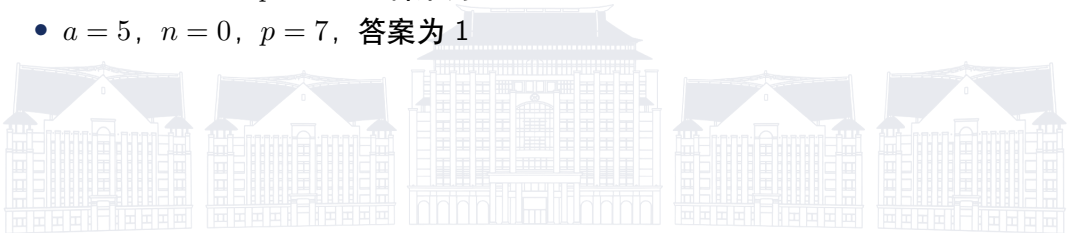
7 总结

例题：快速幂

题目：给定整数 a、n、p，计算 $a^n \bmod p$，其中 n 可以很大。

示例

- $a = 2$，$n = 10$，$p = 1000$，答案为 24
- $a = 3$，$n = 5$，$p = 100$，答案为 43
- $a = 5$，$n = 0$，$p = 7$，答案为 1



快速幂：解法（以 3^{13} 为例）

核心思路：把指数拆成二进制，用“底数平方”预先算好每一幂次，再按位相乘。

关键步骤

- **拆指数**：$13 = 8 + 4 + 1$，所以 $3^{13} = 3^8 \times 3^4 \times 3^1$
- **备幂次**：$3,\ 9,\ 81,\ 6561$，每步平方一次
- **按位乘**：n 末位为 1 就乘，为 0 就跳过，然后 n 右移

为什么正确：指数相加 $=$ 底数相乘，二进制拆分保证每个 3^{2^j} 只乘一次。

指数拆二进制，底数不断平方

快速幂：走一遍 3^{13}

每一步只做三件事：看 n 的末位、必要时把当前 a 乘进答案、a 平方并把 n 右移。

步骤	当前 n	n 末位	当前 a	操作后 ans
初始	13	-	3	1
第 1 步	13	1	3	乘入，$ans = 3$
第 2 步	6	0	9	跳过，$ans = 3$
第 3 步	3	1	81	乘入，$ans = 243$
第 4 步	1	1	6561	乘入，$ans = 1594323$

校验：$3^{13} = 1594323$，与表格结果一致。

观察：指数 13 只需要走 4 步，而不是 13 步。

快速幂：三个变量各管什么

变量	含义	每一步怎么变
ans	答案累乘器，初值 1	末位为 1 时乘上 a，否则不动
a	当前这一位对应的 a^{2^j}	每步平方一次
n	还没处理的指数	每步右移一位，直到为 0

为什么这样能行

- 二进制拆分保证 n 的每一位只对应一个 a^{2^j}
- 末位为 1 表示“这一位要取”，乘入 ans；为 0 表示“不取”，跳过
- a 每步平方，正好走到下一位的 $a^{2^{j+1}}$
- n 右移，末位换到下一位，直到 $n = 0$ 结束

看末位、平方、右移，三步一轮循环

快速幂：复杂度

朴素做法：循环 n 次累乘，时间复杂度 $O(n)$。

快速幂：只循环 n 的二进制位数，时间复杂度 $O(\log n)$。

对比

n 的规模	朴素乘法次数	快速幂乘法次数
10^3	1000	约 10
10^6	1000000	约 20
10^{18}	无法完成	约 60

结论：n 越大，快速幂的优势越明显。

指数再大，也只走二进制位数那么多步

快速幂代码


```
long long fast_pow(long long a, long long n, long long p) {
    long long ans = 1;
    a = a % p;
    while (n > 0) {
        if (n & 1) ans = ans * a % p;     // 末位为 1: 把当前 a 乘入答案
        a = a * a % p;                    // a 平方, 准备下一位
        n = n >> 1;                       // n 右移, 末位换到下一位
    }
    return ans;
}
```

目录

1 倍增思想

2 快速幂

3 线性跳表

4 树上倍增

5 ST 表

6 例题精讲

目录

7 总结

例题：线性跳表

题目：给定长度为 n 的数组 nxt，$nxt[i]$ 表示从位置 i 跳一步到达的位置。多次询问：从位置 x 出发，跳 k 步后到达哪个位置。

示例

- $nxt = [0, 2, 3, 1, 5, 4]$，下标从 1 开始
- 从 $x = 1$ 跳 $k = 2$ 步：$1 \to 2 \to 3$，答案为 3
- 从 $x = 4$ 跳 $k = 1$ 步：$4 \to 5$，答案为 5
- 从 $x = 3$ 跳 $k = 0$ 步：答案为 3

线性跳表：解法

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

四步法

- 状态：$up[i][j]$ 表示从位置 i 跳 2^j 步到达的位置
- 转移：$up[i][j] = up[\,up[i][j-1]\,][j-1]$
- 边界：$up[i][0] = nxt[i]$
- 顺序：j 从 1 到 $\lfloor \log_2 n \rfloor$，i 从 1 到 n

查询：把 k 二进制拆分，逐位跳。

- 若 k 的第 j 位为 1，则 $x \leftarrow up[x][j]$
- 跳完所有位，x 就是答案

预处理 $O(n \log n)$，单次查询 $O(\log n)$

线性跳表代码


```cpp
const int MAXN = 100005;
const int LOG = 17;
int nxt[MAXN];
int up[MAXN][LOG + 1];
void build(int n) {
    for (int i = 1; i <= n; ++i) up[i][0] = nxt[i];
    for (int j = 1; j <= LOG; ++j)
        for (int i = 1; i <= n; ++i)
            up[i][j] = up[up[i][j - 1]][j - 1];
}
int jump(int x, int k) {
    for (int j = 0; j <= LOG; ++j)
        if ((k >> j) & 1) x = up[x][j];
    return x;
}
```

目录

1 倍增思想

2 快速幂

3 线性跳表

4 树上倍增

5 ST 表

6 例题精讲

目录

7 总结

树上倍增：基本思路

场景：一棵有根树，每个节点有唯一的父节点，根节点的父节点指向自己。

核心定义：$up[i][j]$ 表示节点 i 的第 2^j 个祖先。

边界：$up[i][0] = fa[i]$，即节点 i 的父节点。

转移：$up[i][j] = up[\,up[i][j-1]\,][j-1]$。

直观理解：向上跳 2^j 步，等于先跳 2^{j-1} 步，再跳 2^{j-1} 步。

两步 2^{j-1} 拼成一步 2^j

例题：第 k 个祖先

题目：给定一棵有根树，根为节点 1。多次询问：节点 u 的第 k 个祖先是谁。

树结构

节点 i	1	2	3	4	5	6	7
父节点 $fa[i]$	1	1	1	2	2	3	3
深度 $dep[i]$	0	1	1	2	2	2	2

示例

- $u = 7$，$k = 0$，答案为 7
- $u = 7$，$k = 1$，答案为 3
- $u = 7$，$k = 2$，答案为 1

第 k 个祖先：解法

四步法

- 状态：$up[i][j]$ 表示节点 i 的第 2^j 个祖先
- 转移：$up[i][j] = up[\,up[i][j-1]\,][j-1]$
- 边界：$up[i][0] = fa[i]$，根的父节点指向自己
- 顺序：j 从 1 到 $\lfloor \log_2 n \rfloor$，i 从 1 到 n

查询：把 k 二进制拆分，逐位跳。

- 若 k 的第 j 位为 1，则 $u \leftarrow up[u][j]$
- 跳完所有位，u 就是答案

与线性跳表同构：载体从数组换成树

第 k 个祖先代码


```cpp
const int MAXN = 100005;
const int LOG = 17;
int fa[MAXN];
int up[MAXN][LOG + 1];
void build(int n) {
    for (int i = 1; i <= n; ++i) up[i][0] = fa[i];
    for (int j = 1; j <= LOG; ++j)
        for (int i = 1; i <= n; ++i)
            up[i][j] = up[up[i][j - 1]][j - 1];
}
int kth_ancestor(int u, int k) {
    for (int j = 0; j <= LOG; ++j)
        if ((k >> j) & 1) u = up[u][j];
    return u;
}
```

目录

1 倍增思想

2 快速幂

3 线性跳表

4 树上倍增

5 ST 表

6 例题精讲

目录

7 总结

区间最值问题

问题：给定长度为 n 的数组 a，多次询问区间 $[l, r]$ 内的最大值。

两种朴素做法

- 每次扫描区间：单次 $O(n)$，q 次询问 $O(nq)$
- 预处理所有区间：$O(n^2)$ 时间与空间

目标：预处理 $O(n \log n)$，单次查询 $O(1)$。

预处理加倍增，查询常数时间

ST 表：状态定义

核心定义：$st[i][j]$ 表示从位置 i 开始、长度为 2^j 的区间内的最大值。

区间范围：$[i,\ i+2^j-1]$。

基础层：$j=0$ 时，区间长度 $2^0=1$，$st[i][0]=a[i]$。

参数范围

- $1 \le i \le n$
- $0 \le j \le \lfloor \log_2 n \rfloor$

直观理解：用长度翻倍的区间去覆盖整个数组，每一层都是上一层信息的合并。

ST 表：转移方程

关键观察：长度 2^j 的区间，可以拆成两个长度 2^{j-1} 的区间。

$$st[i][j] = \max\left(st[i][j-1],\ st[i+2^{j-1}][j-1]\right)$$

拆解

- 左半块：从 i 开始，长度 2^{j-1}
- 右半块：从 $i+2^{j-1}$ 开始，长度 2^{j-1}
- 两块的并集正好是 $[i,\ i+2^j-1]$

递推顺序：先从小到大枚举 j，再枚举 i，因为第 j 层依赖第 $j-1$ 层。

复杂度：状态数 $n\log n$，转移 $O(1)$，总时间 $O(n\log n)$。

ST 表：查询

查询 $[l, r]$：**设长度** $len = r - l + 1$，**令** $k = \lfloor \log_2 len \rfloor$。

两块覆盖：**用两个长度为** 2^k **的块覆盖** $[l, r]$。

- **左块**：**从** l **开始，长度** 2^k，**即** $st[l][k]$
- **右块**：**从** $r - 2^k + 1$ **开始，长度** 2^k，**即** $st[r - 2^k + 1][k]$

答案：$\max(st[l][k],\ st[r - 2^k + 1][k])$。

为什么允许重叠：**最大值可重复合并，重叠部分不影响结果。**

两块覆盖，允许重叠

对数表的预处理

为什么需要 lg 数组：查询时要快速求 $\lfloor \log_2 len \rfloor$，不能每次调用浮点函数。

递推式：$lg[i] = lg[i/2] + 1$，$lg[1] = 0$。

正确性

- 除以 2 相当于右移一位
- 每右移一位，$\log_2$ 的结果恰好减少 1
- 递推到 $i = 1$ 时停止，$lg[1] = 0$

复杂度：$O(n)$ 预处理，查询时 $O(1)$ 取值。

用递推求对数

ST 表代码


```cpp
const int MAXN = 100005;
const int LOG = 17;
int st[MAXN][LOG + 1];
int lg[MAXN];
void build_st(int n, int a[]) {
    lg[1] = 0;
    for (int i = 2; i <= n; ++i) lg[i] = lg[i >> 1] + 1;
    for (int i = 1; i <= n; ++i) st[i][0] = a[i];
    for (int j = 1; j <= LOG; ++j) {
        for (int i = 1; i + (1 << j) - 1 <= n; ++i) {
            st[i][j] = max(st[i][j - 1], st[i + (1 << (j - 1))][j - 1]);
        }
    }
}
int query(int l, int r) {
    int k = lg[r - l + 1];
    return max(st[l][k], st[r - (1 << k) + 1][k]);
}
```

目录

1 倍增思想

2 快速幂

3 线性跳表

4 树上倍增

5 ST 表

6 例题精讲

目录

7 总结

例题：区间最大值

题目：给定长度为 n 的数组 a，共 q 次询问，每次给出 l 和 r，输出 $a[l], a[l+1], \ldots, a[r]$ 中的最大值。

示例

- $a = [3, 1, 4, 1, 5, 9, 2, 6]$
- 询问 $[1, 5]$，答案为 5
- 询问 $[3, 8]$，答案为 9
- 询问 $[6, 6]$，答案为 9

区间最大值：解法

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

四步法

- **状态**：$st[i][j]$ 表示从 i 开始、长度 2^j 的区间最大值
- **转移**：$st[i][j] = \max(st[i][j-1], st[i+2^{j-1}][j-1])$
- **边界**：$st[i][0] = a[i]$
- **顺序**：j 从 1 到 $\lfloor \log_2 n \rfloor$，i 从 1 到 $n - 2^j + 1$

查询 $[l, r]$：令 $k = \lfloor \log_2(r - l + 1) \rfloor$，答案为 $\max(st[l][k],\ st[r - 2^k + 1][k])$。

两块覆盖，允许重叠；预处理 $O(n \log n)$，查询 $O(1)$

例题：区间 gcd

题目：给定长度为 n 的数组 a，共 q 次询问，每次给出 l 和 r，输出 $a[l], a[l+1], \ldots, a[r]$ 的最大公约数。

示例

- $a = [12, 18, 24, 30, 36]$
- **询问** $[1, 3]$：$\gcd(12, 18, 24) = 6$
- **询问** $[2, 5]$：$\gcd(18, 24, 30, 36) = 6$
- **询问** $[1, 5]$：$\gcd(12, 18, 24, 30, 36) = 6$

区间 gcd：解法

四步法

- **状态**：$st[i][j]$ 表示从 i 开始、长度 2^j 的区间 gcd
- **转移**：$st[i][j] = \gcd(st[i][j-1], st[i+2^{j-1}][j-1])$
- **边界**：$st[i][0] = a[i]$
- **顺序**：j 从 1 到 $\lfloor \log_2 n \rfloor$，i 从 1 到 $n - 2^j + 1$

查询 $[l, r]$：令 $k = \lfloor \log_2(r - l + 1) \rfloor$，答案为 $\gcd(st[l][k],\ st[r - 2^k + 1][k])$。

把 max 换成 gcd，其余完全一样

目录

1 倍增思想

2 快速幂

3 线性跳表

4 树上倍增

5 ST 表

6 例题精讲

目录

⑦ 总结

总结（一）：倍增思想

倍增法的两个要素

- 二进制拆分：任意整数可拆成 2 的幂之和
- 可合并性：小区间答案可拼成大区间答案

典型应用

- 快速幂：底数不断平方，指数不断右移
- 线性跳表：$up[i][j]$ 表示从 i 跳 2^j 步
- 树上倍增：$up[i][j]$ 表示节点 i 的第 2^j 个祖先
- ST 表：区间长度翻倍，逐层合并

凡是能二进制拆分的规模，都能用倍增加速

总结（二）：跳表与树上倍增

项目	线性跳表	树上倍增
状态 $up[i][0]$	$nxt[i]$	$fa[i]$
转移	$up[up[i][j-1]][j-1]$	$up[up[i][j-1]][j-1]$
查询	k 二进制拆分逐位跳	k 二进制拆分逐位跳
预处理	$O(n \log n)$	$O(n \log n)$
单次查询	$O(\log n)$	$O(\log n)$

易错点

- 根节点的父节点要指向自己，避免跳过头
- 查询时 k 的二进制位要从低到高逐位处理
- 数组下标从 1 开始还是从 0 开始，全文要统一

总结（三）：ST 表要点

项目	说明
预处理时间	$O(n \log n)$
单次查询	$O(1)$
空间复杂度	$O(n \log n)$
适用条件	可重复贡献（max、min、gcd 等）
不适用	区间和、区间众数

易错点

- 忘记预处理 lg 数组，查询时直接调用浮点函数
- 转移时下标越界，需保证 $i + 2^j - 1 \le n$

倍增建表，对数查询，两块覆盖

附录：常见陷阱

快速幂

- 底数未先取模，导致溢出
- 循环条件写成 $n > 1$，遗漏 $n = 1$ 的情况

线性跳表与树上倍增

- 根节点的 $up[root][0]$ 未设为自身，跳祖先时越界
- 查询时 k 的二进制位处理顺序写反，导致跳错

ST 表

- LOG 取值过小，数组越界
- 转移时没有判断右端点，访问到不存在的下标
- 查询时 k 算错，覆盖区间不完整